

React Interview Questions

1. What is flux in react

What is Flux?

Flux is an application architecture (pattern), not a library, introduced by Facebook to manage data flow in React applications.

- 📌 React handles UI,
 - 📌 Flux handles how data moves and changes in the app.
-

Why Flux was introduced?

Before Flux:

- Apps had two-way data binding
- Data changes were hard to track
- Bugs were difficult to debug

👉 Flux enforces one-way (unidirectional) data flow, making apps predictable.

Core Idea of Flux (Very Important)

Data flows in only one direction

Action → Dispatcher → Store → View (React) → Action

Main Components of Flux

1 Action

- Plain JavaScript object
- Describes what happened

```
{  
  type: "ADD_TODO",  
  payload: "Learn React"  
}
```

📌 Actions do NOT change data directly.

2 Dispatcher

- Central hub
- Receives actions
- Sends them to all stores

📌 There is only one dispatcher in a Flux app.

3 Store

- Holds application state
- Contains business logic
- Updates state based on action type

```
if (action.type === "ADD_TODO") {  
  // update store state  
}
```

 Stores decide how data changes

4 View (React Components)

- Displays data from store
- Triggers actions on user interaction

```
<button onClick={addTodo}>Add</button>
```

Complete Flux Flow (Easy Example)

Step-by-step:

- 1 User clicks button
- 2 View creates an Action
- 3 Dispatcher sends action
- 4 Store updates state
- 5 Store notifies React
- 6 UI re-renders

 Flow repeats

Why Flux is useful?

- Predictable data flow
- Easier debugging
- Better scalability

- Clear separation of concerns
-

Flux vs MVC (Important Interview Point)

MVC	Flux
Two-way data binding	One-way data flow
Hard to debug	Easy to debug
Model ↔ View	Store → View only

Is Redux the same as Flux?

👉 Redux is inspired by Flux, but simpler.

Flux	Redux
Multiple stores	Single store
Dispatcher	No dispatcher
Mutable state	Immutable state

Do we still use Flux today?

- Pure Flux ❌ (rare)
- Redux / Zustand / Recoil ✅

- Concept of Flux is still VERY important
-

Interview-ready answer ★

Flux is an architectural pattern used with React to manage application state using unidirectional data flow, making data changes predictable and easier to debug.

What is Two-Way Data Binding?

Two-way data binding means:

UI can change data AND data can change UI automatically

So the data flow goes in **both directions**.

View ⇌ Model

Simple Example (Easy to understand)

Without React (Two-way binding – AngularJS style)

```
<input ng-model="name">
<p>{{ name }}</p>
```

What happens?

- User types in input → **name** changes
- **name** changes → UI updates automatically

📌 Data and UI are **directly connected** both ways.

Why Two-Way Binding Causes Problems?

1 Hard to track changes

If UI changes data:

- Which component changed it?
- From where did the update come?

👉 Debugging becomes difficult

2 Unpredictable state

- Multiple views can update the same data
- Data changes from many directions

👉 Leads to bugs in large applications

3 Tight coupling

- Model and View are tightly connected
 - Changing one affects the other unexpectedly
-

Visual Problem

Component A ⇌ Data ⇌ Component B ⇌ Component C

👉 One change can trigger many hidden updates.

How Flux Solves This Problem

Flux introduces **one-way (unidirectional) data flow**:

Action → Dispatcher → Store → View

📌 Now:

- UI cannot directly change data
 - Data changes happen only in **Store**
 - Flow is **predictable**
-

React & Data Binding

React uses:

- **One-way data binding**
- Parent → Child data flow

```
<input value={name} onChange={handleChange} />
```

📌 Even though input looks two-way, it is actually:

- Event → State update → Re-render
-

Interview-ready answer ★

Two-way data binding allows automatic synchronization between the UI and the data model in both directions, but it makes large applications hard to debug and maintain.

2. Context API

What is Context API?

Context API is a React feature used to **share data globally** between components **without passing props manually at every level** (avoids *prop drilling*).

The Problem: Prop Drilling ❌

App → Parent → Child → GrandChild

If **App** has data (like **user**), you must pass it through every component even if they don't need it.

👉 This is called **prop drilling**.

Solution: Context API ✅

Context lets you:

- Create **global data**
- Access it directly where needed

Provider → Any Component

How Context API Works (3 Steps)

1 Create Context

```
const UserContext = React.createContext();
```

2 Provide Context

```
<UserContext.Provider value="Vishal">  
  <App />  
</UserContext.Provider>
```

 **Provider** makes data available to all children.

3 Consume Context

```
const user = useContext(UserContext);
```

Complete Easy Example

Context File

```
import { createContext } from "react";

export const ThemeContext = createContext();
```

Provider

```
function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Navbar />
    </ThemeContext.Provider>
  );
}
```

Consumer

```
import { useContext } from "react";
import { ThemeContext } from "../ThemeContext";

function Navbar() {
  const theme = useContext(ThemeContext);
  return <h1>Theme: {theme}</h1>;
}
```

When should you use Context API?

- Theme (dark/light)
- Authentication (user data)
- Language settings
- App-wide configuration

When NOT to use Context API

- For frequently changing data
- Large, complex state logic

👉 Use **Redux / Zustand** instead

Context API vs Redux (Interview Point)

Context API	Redux
Built-in React	External library
Simple global data	Complex global state

No middleware	Middleware support
Re-renders more	Optimized

Important Interview Notes ★

- Context is **NOT** a state manager
 - It is a **dependency injection system**
 - Frequent updates cause re-renders
-

Interview-ready answer

Context API allows sharing data globally in a React app without prop drilling by using Providers and Consumers.

What is React Router?

React Router is a **library** used in React applications to handle **navigation and routing** in **Single Page Applications (SPA)**.

- 📌 It allows you to show **different components** based on the **URL**
 - 📌 Page does **not reload**
-

Why do we need React Router?

Without React Router:

- React app has only **one page**

- Changing URL reloads the whole page

With React Router:

- URL changes
 - Only required component updates
 - App feels fast (SPA behavior)
-

What is Routing?

Routing means mapping **URLs (paths)** to **components**.

Example:

```
/login → Login component  
/about → About component
```

Core Components of React Router

1 BrowserRouter

- Wraps the entire app
- Uses HTML5 history API

```
<BrowserRouter>  
  <App />  
</BrowserRouter>
```

2 Routes & Route

- Define paths and components

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
</Routes>
```

3 Link

- Used for navigation
- Prevents page reload

```
<Link to="/about">About</Link>
```

4 useParams

- Access URL parameters

```
const { id } = useParams();
```

URL:

```
/user/5
```

5 useNavigate

- Navigate programmatically

```
const navigate = useNavigate();
```

```
navigate("/login");
```

Simple Example (Easy)

```
import { BrowserRouter, Routes, Route, Link } from
"react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

React Router v6 (Interview ★)

Important changes:

- <Switch> ❌ → <Routes> ✅
- component ❌ → element
- Exact match by default

What is **BrowserRouter**?

BrowserRouter is a **router component** provided by **React Router** that:

- 1 Wraps your entire application
 - 2 Uses the **HTML5 History API**
 - 3 Syncs the **URL** with **React components**
 - 4 Enables **client-side routing (SPA)**
-

Basic Usage

```
import { BrowserRouter } from "react-router-dom";  
  
<BrowserRouter>  
  <App />  
</BrowserRouter>
```

 This makes routing available to **all child components**.

What does “Uses HTML5 History API” mean?

HTML5 History API provides:

- `pushState()`
- `replaceState()`
- `popstate` event

These allow:

- Changing URL **without page reload**

- Navigating back/forward using browser buttons

How Routing Works Internally (VERY IMPORTANT)

Let's break it step by step 📌

Step **1** Initial Page Load

URL in browser:

`http://localhost:3000/about`

Browser:

- Requests `index.html`
- React app loads **once**

📌 Server always returns the same HTML file.

Step **2** BrowserRouter Reads the URL

`window.location.pathname`

Example:

`/about`

BrowserRouter:

- Reads current path
 - Stores it in its internal state
-

Step 3 Routes Match the Path

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
</Routes>
```

React Router:

- Compares current URL (`/about`)
- Finds matching `<Route>`
- Renders `<About />`

 Only **component changes**, page does NOT reload.

Step 4 Navigation using `<Link>`

```
<Link to="/contact">Contact</Link>
```

Internally:

```
history.pushState({}, "", "/contact");
```

What happens?

- URL changes
 - BrowserRouter detects change
 - Matching component renders
 - No page reload
-

Step Browser Back / Forward Buttons

When user clicks:

-  Back
-  Forward

Browser triggers:

`window.onpopstate`

BrowserRouter:

- Listens to this event
 - Updates route
 - Renders correct component
-

Complete Internal Flow (Diagram)

URL change



BrowserRouter listens

↓
Matches Route
↓
Renders Component

Why wrap the entire app?

Because:

- Any component may need routing
- Hooks like `useNavigate`, `useParams` need Router context
- Only ONE router should exist

✗ Without `BrowserRouter` → routing breaks

What is HTML5 History API?

The **HTML5 History API** allows JavaScript to:

- Change the browser URL
- Navigate history
- **WITHOUT** reloading the page

📌 This is the backbone of **Single Page Applications (SPA)** like React.

1 `pushState()`

What does it do?

- Adds a **new entry** to browser history
- Changes the URL
- **Does NOT reload the page**

Syntax

```
history.pushState(state, title, url);
```

Example

```
history.pushState({}, "", "/about");
```

What happens internally?

- URL changes to `/about`
- Browser history stack gets a new entry
- `popstate` event **✗** NOT triggered
- React Router manually updates UI

 Used when navigating normally (Link, navigate)

2 `replaceState()`

What does it do?

- Replaces the **current history entry**
- URL changes
- **Does NOT add a new history record**

Syntax

```
history.replaceState(state, title, url);
```

Example

```
history.replaceState({}, "", "/login");
```

Use Case

- Redirect after login
- Prevent going back to previous page

📌 Back button will skip replaced page

3 popstate Event

What is it?

- Event fired when:
 - User clicks Back / Forward
 - `history.back()` or `history.forward()` is called

Example

```
window.addEventListener("popstate", () => {  
  console.log("URL changed:", window.location.pathname);  
});
```

Important Note !

✗ `popstate` is NOT fired by:

- `pushState()`
 - `replaceState()`
-

How React Router Uses These APIs (BIG PICTURE)

When you click `<Link>`

```
history.pushState({}, "", "/profile");
```

React Router:

- Updates internal route state
 - Matches new URL
 - Renders new component
-

When you redirect

```
history.replaceState({}, "", "/dashboard");
```

React Router:

- Replaces current route
 - Prevents going back
-

When you click **Back** button

Browser:

- Fires `popstate` event

React Router:

- Listens to `popstate`
- Updates UI based on URL

Visual Flow Diagram

Link click → `pushState` → URL changes → React renders

Back button → `popstate` → React renders

Comparison Table (Interview)

API	History Stack	Page Reload	Event Fired
<code>pushState</code>	Adds new	 No	 No
<code>replaceState</code>	Replaces	 No	 No
<code>popstate</code>	Navigation	 No	 Yes

Real-Life Analogy



Train Ticket System

- `pushState()` → Buy new ticket
 - `replaceState()` → Change current ticket
 - `popstate` → Travel backward/forward
-

One-line Interview Answers

- **pushState**: Adds a new URL without reloading
- **replaceState**: Replaces current URL without reloading
- **popstate**: Event fired on browser navigation